

Industrial Oriented Mini Project(CM653PC)

ON

Vision Assistant For Visually Impaired Individuals

Submitted

*In partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science Engineering

(Artificial Intelligence and Machine Learning)

by

Meghana Doodi (23261A6612)

&

Rohith Ranaveni (23261A6641)

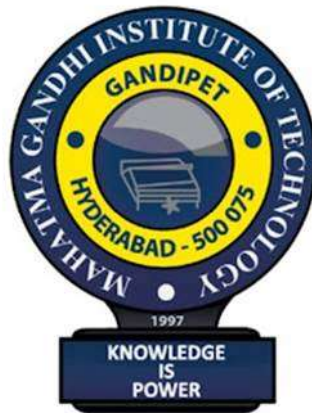
Under the guidance of

Dr. Barnali Gupta Banik

(Associate Professor)

Ms. S. Rajini Parimila

(Assistant Professor)



Department of Emerging Technologies

Mahatma Gandhi Institute of Technology (Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

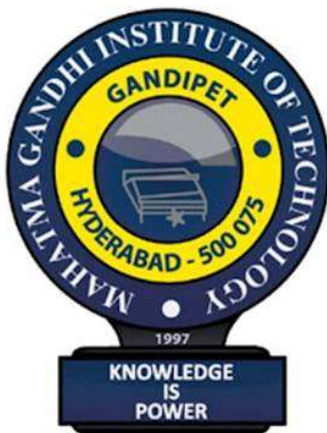
Kokapet(V), Gandipet(M), Hyderabad. Telangana - 500075

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
(Affiliated To Jawaharlal Nehru Technological University Hyderabad)

GANDIPET, HYDERABAD-500075, Telangana

CERTIFICATE



This is to certify that the project entitled —“Vision Assistant For Visually Impaired Individuals” is being submitted by MEGHANA DOODI (23261A6612), ROHITH RANA VENI (23261A6641) in partial fulfillment of the requirements for the Industrial Oriented Mini Project (CM653PC) in COMPUTER SCIENCE AND ENGINEERING (AI & ML) is a record of bonafide work carried out by them under our guidance and supervision.

The results of the investigations enclosed in this report have been verified and found satisfactory.

PROJECT GUIDE

Dr. Barnali Gupta Banik
Associate Professor
Department of ET

Ms. S. Rajini Parimila
Assistant Professor
Department of ET

HEAD OF THE DEPARTMENT

Dr. M. Rama Bai
Professor & Head
Department of ET

EXTERNAL EXAMINER

Declaration

This is to certify that the work reported in this project titled “**Vision Assistant For Visually Impaired Individuals**” is a record of work done by us in the Department of Computer Science and Engineering (Artificial intelligence and Machine Learning),Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

MEGHANA DOODI

(23261A6612)

ROHITH RANA VENI

(23261A6641)

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We would like to express our sincere thanks to **Prof. G. ChandraMohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. M. RamaBai, Professor and HoD**, Department of Emerging Technologies, MGIT, for all the timely support and valuable suggestions during the period of the project.

We are extremely thankful and indebted to our mentors **Dr. Barnali Gupta Banik , Associate Professor, Ms. S. Rajini Parimila, Assistant professor**, Department of Emerging Technologies, for their constant guidance, encouragement and moral support throughout the project.

We are extremely thankful to **Ms. J. Sreedevi, Assistant professor, Ms. Asha Jyothi Assistant professor** Department of Emerging Technologies, MGIT, IOMP project coordinator for her encouragement and support throughout the project.

Finally, we would also like to thank all the faculty and staff of the Emerging Technologies Department who helped us directly or indirectly to complete this project.

MEGHANA DOODI

(23261A6612)

ROHITH RANA VENI

(23261A6641)

Table of Content

TOPIC	PAGE NO.
Certificate	I
Declaration	III
Acknowledgement	IV
List of Figures	VII
List of Tables	VIII
Abstract	IX
1 Introduction	1
1.1 Motivation	
1.2 Problem Definition	
1.3 Existing System	
1.4 Proposed System	
1.5 Requirements Specification	
2 Literature Survey	7
3 Design and Methodology	11
3.1 System Overview	
3.2 Dataset Design and Preparation	
3.3 Voice Processing Module	
3.4 Data Processing and Classification	
3.5 User Interface Integration	

4 Implementation and Results	14
4.1 Overview	
4.2 Front-End Components	
4.3 Back-End Components	
4.4 Session State Management	
4.5 Key Features Implemented	
4.6 Sample Results	
4.7 Result Analysis	
5 Conclusion and Future Scope	21
Bibliography	23
Appendix	24

List of Figures

FIGURE No.	FIGURE NAME	PAGE No.
Figure 4.7.1	The figure shows two modes: Image Captioning and Live Video Description	18
Figure 4.7.2	The figure shows the language selection screen with options for Telugu, Hindi, and English	19
Figure 4.7.3	The figure shows the camera screen capturing a laptop, where the app processes the image to generate a description	20

List of Tables

TABLE No.	TABLE NAME	PAGE No.
Table 2.1	Literature Survey Summary	10
Table 5.2.1	Key Front-End Components	15
Table 5.3.1	Key Back-End Components	16
Table 5.5.1	Key Features Implemented	17
Table 5.7.1	Result Analysis	18

ABSTRACT

The Vision Assistant App is an AI-based mobile application developed to assist visually impaired individuals by providing real-time understanding of their surroundings. The system uses a smartphone camera to capture images and applies deep learning techniques to generate meaningful captions describing the scene. These captions are then converted into speech, enabling users to perceive and interpret their environment through audio feedback.

The application is built using a Flutter-based frontend and a Flask backend, where a pre-trained image captioning model processes the captured images. In addition to scene description, the system includes a hazard detection feature that identifies potentially dangerous objects such as vehicles, fire, or sharp items and alerts the user through voice warnings. Furthermore, the application supports multilingual output, allowing users to receive descriptions in languages such as English, Hindi, and Telugu.

By integrating artificial intelligence, computer vision, and mobile technologies, the Vision Assistant App provides an efficient and user-friendly assistive solution. The optimized system ensures reduced delay and near real-time performance, making it suitable for everyday use. This project demonstrates how AI-driven solutions can enhance accessibility, improve safety, and promote independence for visually impaired individuals.

Keywords: Artificial Intelligence, Computer Vision, Image Captioning ,Text-to-Speech, Multilingual Support, Real-Time Processing, and Mobile Application

1. INTRODUCTION

1.1 Introduction to the Project

The Vision Assistant App is an artificial intelligence-based mobile application designed to assist visually impaired individuals by providing real-time understanding of their surroundings. With the rapid advancement of computer vision and deep learning technologies, it has become possible for machines to interpret visual information and convert it into meaningful descriptions. This project leverages such technologies to capture images through a smartphone camera and generate descriptive captions, which are then converted into speech for user accessibility.

Visually impaired individuals often face difficulties in identifying objects, understanding environments, and navigating safely in their daily lives. Traditional assistive tools provide limited support and may not offer real-time information about the surroundings. The Vision Assistant App aims to address these challenges by acting as an intelligent companion that can describe scenes, identify objects, and provide audio feedback instantly. This enhances independence and improves the quality of life for users.

The system is built using a combination of modern technologies, including a Flutter-based mobile application and a Python Flask backend. The frontend handles camera operations, user interaction, and audio output, while the backend processes images using a pre-trained deep learning model for image captioning. The generated captions are further translated into multiple languages such as English, Hindi, and Telugu, making the system more inclusive and accessible to a wider audience.

The working process of the application involves capturing an image, sending it to the backend server, processing it using artificial intelligence, and returning a descriptive caption. The caption is then translated and converted into speech using a text-to-speech module. The system also supports live mode, where the process repeats continuously to provide near real-time assistance. The design ensures minimal delay and efficient performance through optimized models and processing techniques.

One of the key strengths of the Vision Assistant App is its integration of multiple functionalities into a single platform. It combines image captioning, language translation, and speech synthesis to deliver a complete assistive solution. The system is designed to be lightweight, user-friendly, and accessible through a standard smartphone, eliminating the need for expensive or specialized hardware.

In conclusion, the Vision Assistant App demonstrates the practical application of artificial intelligence in assistive technology. By integrating computer vision, natural language processing, and mobile development, the project provides an effective and scalable solution for visually impaired users. It highlights how modern AI technologies can be used to improve accessibility, enhance safety, and promote independence in everyday life.

1.2 Existing System

Existing systems designed to assist visually impaired individuals mainly include tools such as screen readers, basic object detection devices, GPS-based navigation systems, and wearable assistive technologies. These systems provide limited functionality by helping users read text, detect specific objects, or navigate from one location to another. Some advanced systems use computer vision techniques, but they are often complex and require specialized hardware, making them less accessible to common users.

Many existing solutions focus only on a single feature, such as object detection or text reading, and do not provide a complete understanding of the surrounding environment. For example, object detection systems can identify individual objects but cannot generate meaningful descriptions of an entire scene. Similarly, navigation systems help in movement but do not provide information about nearby objects or activities.

Another limitation of existing systems is their lack of real-time performance and multilingual support. Some systems have high processing delays, while others only support a single language, reducing their usability for diverse users. Additionally, many assistive devices are expensive and not easily portable, which makes them impractical for everyday use.

Therefore, although existing systems provide basic assistance, they fail to offer a comprehensive, real-time, and user-friendly solution that combines scene understanding, language flexibility, and audio feedback in a single mobile application.

1.3 Problem Definition

Visually impaired individuals face significant challenges in understanding and interacting with their surroundings. They often rely on external assistance or limited tools to identify objects, navigate environments, and perform daily activities. Existing assistive technologies provide only partial solutions, such as reading text or detecting specific objects, but they do not offer a complete understanding of the scene in real time.

There is a need for an intelligent system that can capture visual information, analyze it, and provide meaningful descriptions in an accessible format. The system should be capable of generating accurate scene descriptions, supporting multiple languages, and delivering output through audio to ensure usability for visually impaired users. Additionally, it should be portable, cost-effective, and easy to use on commonly available devices like smartphones.

Therefore, the main problem is to design and develop a real-time, AI-based assistive application that can interpret visual data, generate descriptive captions, and provide multilingual audio feedback with minimal delay, thereby improving independence and safety for visually impaired individuals.

OBJECTIVES

The specific objectives of the project are:

1. To capture images using a smartphone camera and process them efficiently.
2. To generate accurate and meaningful captions using deep learning models.
3. To convert the generated text into speech for easy understanding.
4. To provide multilingual support such as English, Hindi, and Telugu.
5. To reduce processing delay and achieve near real-time performance.
6. To design a simple and accessible user interface for visually impaired users.
7. To develop a cost-effective solution that works on standard mobile devices.

1.4 Proposed System

The proposed system is the Vision Assistant App, an AI-based mobile application designed to provide real-time assistance to visually impaired individuals. The system uses a smartphone camera to capture images and applies deep learning techniques to generate descriptive captions of the surrounding environment. These captions are then converted into speech, enabling users to understand their surroundings through audio feedback.

The application is developed using a Flutter frontend and a Python Flask backend. The frontend is responsible for capturing images, handling user interaction, and providing audio output. The backend processes the captured images using a pre-trained image captioning model (BLIP), which generates meaningful descriptions of the scene. The generated captions are further translated into multiple languages such as English, Hindi, and Telugu to improve accessibility.

The system follows a simple and efficient workflow where the captured image is sent to the backend via an API. The backend processes the image, generates a caption, translates it based on user preference, and sends the result back to the mobile application. The app then converts the text into speech using a text-to-speech module, providing clear audio output to the user.

The proposed system is designed to overcome the limitations of existing systems by providing a complete solution that integrates image captioning, language translation, and speech output into a single platform. It is cost-effective, portable, and easy to use, as it runs on a standard smartphone

without requiring additional hardware. The system also ensures reduced processing delay, enabling near real-time performance.

Overall, the Vision Assistant App offers an efficient and user-friendly assistive solution that enhances independence, safety, and accessibility for visually impaired individuals.

1.5 Requirements Specification

1.5.1 Software Requirements

- **Operating System:** Windows 10 or later / Ubuntu 20.04 LTS / Android OS
- **Programming Languages:** Python 3.9 or later, Dart (Flutter)
- **Frontend Framework:** Flutter (for mobile application development)
- **Backend Framework:** Flask (Python-based REST API)
- **AI & Deep Learning Libraries:** Hugging Face Transformers (BLIP model), PyTorch
- **Image Processing:** PIL (Python Imaging Library), OpenCV (optional)
- **Translation Library:** Deep Translator (Google Translator API)
- **Text-to-Speech:** Flutter TTS Plugin
- **API Communication:** HTTP package (Flutter)
- **Data Processing:** NumPy, basic preprocessing techniques
- **Development Environment:** Android Studio, Visual Studio Code, PyCharm
- **Version Control:** Git, GitHub
- **API Testing:** Postman
- **Deployment:** Localhost / Flask Server / Optional Cloud (AWS / Heroku)

1.5.2 Hardware Requirements

The hardware specifications below are required to support model training, fine-tuning, and real-time inference at acceptable latency.

- **Processor:** Intel i5 or higher / AMD equivalent
- **RAM:** Minimum 8 GB (Recommended: 16 GB for smooth AI processing)
- **Storage:** Minimum 20 GB free space
- **GPU (Optional):** NVIDIA GPU (for faster model processing, optional)
- **Mobile Device:** Android smartphone with camera
- **Camera:** Minimum 8 MP camera (for clear image capture)
- **Internet Connection:** Required for API communication and translation
- **Display:** Standard monitor or laptop screen
- **Input Devices:** Keyboard and mouse

2. LITERATURE SURVEY

Recent advancements in artificial intelligence and computer vision have significantly improved the ability of machines to understand visual data. Early research such as “Show and Tell: A Neural Image Caption Generator” by Vinyals et al. introduced the use of Convolutional Neural Networks (CNN) combined with Recurrent Neural Networks (RNN) for generating image captions. Later, attention-based models like “Show, Attend and Tell” further improved caption accuracy by focusing on important regions of an image.

With the evolution of deep learning, transformer-based models and vision-language models such as BLIP (Bootstrapping Language-Image Pretraining) have achieved higher accuracy and better contextual understanding. These models are capable of generating more meaningful and human-like descriptions compared to earlier approaches. Such advancements have made real-time image captioning more practical for applications like assistive technologies.

In the field of assistive systems, several studies have focused on helping visually impaired individuals using computer vision techniques. Existing solutions include object detection systems, navigation aids, and wearable devices. However, many of these systems either provide limited information or require expensive hardware, making them less accessible for everyday use.

Translation and speech technologies also play an important role in improving accessibility. Neural Machine Translation systems have enabled accurate language conversion, while Text-to-Speech (TTS) systems allow text to be converted into natural-sounding speech. These technologies help in delivering information in a more user-friendly manner.

Despite these advancements, there is still a need for an integrated system that combines image captioning, translation, and audio output into a single mobile application. The proposed Vision Assistant App addresses this gap by providing a unified solution that is portable, efficient, and easy to use. It leverages modern AI models and mobile technologies to deliver real-time assistance and improve the independence of visually impaired users.

LITERATURE SURVEY SUMMARY

This table summarizes key research works, methodologies, and findings related to image captioning and assistive technologies.

S.No	Author(s)	Year	Title	Methodology / Technology Used	Key Findings
1	Various Researchers	2021–2024	AI-based Assistive Systems	Deep Learning + Mobile Apps	Demonstrated real-time assistance for visually impaired users.
2	Li et al.	2023	BLIP-2	Advanced Vision-Language Model	Improved performance by combining vision encoders with language models.
3	Li et al.	2022	BLIP: Bootstrapping Language-Image Pretraining	Vision-Language Model	High accuracy image captioning and multimodal understanding.
4	Radford et al.	2021	CLIP	Vision-Language Model	Strong image-text understanding across tasks.
5	Cornia et al.	2020	Meshed-Memory Transformer	Transformer Architecture	Enhanced contextual understanding in captions.

6	Bochkovski et al.	2020	YOLOv4	Improved CNN	Better accuracy and speed for object detection.
7	Redmon & Farhadi	2018	YOLOv3	CNN (YOLO)	Real-time object detection with high speed.
8	Anderson et al.	2018	Bottom-Up and Top- Down Attention	Faster R- CNN + Attention	Improved object- level captioning accuracy.
9	Howard et al.	2017	MobileNets	Lightweight CNN	Optimized models for mobile devices.
10	Xu et al.	2016	Show, Attend and Tell	CNN + Attention RNN	Improved caption accuracy using attention mechanism.
11	Wu et al.	2016	Google Neural Machine Translation	Neural Networks	High-quality language translation system.
12	Vinyals et al.	2015	Show and Tell	CNN + RNN	Generated image captions using deep learning techniques.

13	Karpathy & Fei-Fei	2015	Deep Visual-Semantic Alignments	CNN + LSTM	Linked image features with textual descriptions effectively.
14	Manduchi et al.	2010	Assistive Technology for Visually Impaired	Computer Vision	Early work on navigation and assistive systems.
15	Taylor	2009	Text-to-Speech Synthesis	TTS Systems	Converts text into natural speech output.

Table 2.1: Literature Survey Summary

3. DESIGN AND METHODOLOGY

The Vision Assistant App is designed using a modular and structured approach to provide real-time assistance to visually impaired users. The system follows a client-server architecture, where the mobile application acts as the frontend and communicates with a backend server for processing. This separation allows efficient handling of tasks and ensures scalability and maintainability of the system.

The frontend of the application is developed using Flutter, which provides a user-friendly interface and handles camera functionality, user interaction, and audio output. The backend is implemented using Python and Flask, where artificial intelligence models are used for image processing and caption generation. The communication between the frontend and backend is achieved using REST APIs.

The system takes input in the form of images captured through the smartphone camera. The image capture process is optimized by selecting an appropriate resolution to balance quality and performance. Once the image is captured, it is sent to the backend server for processing through an API request.

On the backend, the image is processed using a pre-trained deep learning model (BLIP) to generate a descriptive caption. The generated caption represents the scene in a meaningful way. After caption generation, the text is translated into the selected language, enabling multilingual support for users.

The output of the system is provided as audio feedback. The translated caption is converted into speech using a text-to-speech module, allowing users to hear and understand their surroundings. The user interface is designed to be simple and accessible, with features like camera preview and language selection.

To improve performance, the system uses optimization techniques such as lightweight models, reduced caption length, and a single API call. These improvements help in reducing delay and achieving near real-time performance. Overall, the design ensures efficiency, accuracy, and ease of use for visually impaired users. The figure below illustrates the overall workflow of the Vision Assistant App, showing how input images are processed to generate audio output.



Fig. no. 3.1: The overall workflow of the Vision Assistant App

The flowchart represents the working of the Vision Assistant App. The system captures an image, sends it to the backend, and generates a caption using an AI model. The caption is translated into the selected language and converted into speech. If live mode is enabled, the process repeats continuously; otherwise, it ends.

3.1 System Overview

The system is a mobile-based application developed using Flutter (frontend) and Python Flask (backend). It captures images, processes them using AI, and provides audio output. The app helps users understand their surroundings in real time.

3.2 Dataset Design and Preparation

The system uses pre-trained models trained on large image datasets. It does not require a custom dataset, as it directly processes real-time images captured by the user.

3.3 Voice Processing Module

This module is responsible for capturing images using the smartphone camera. It is implemented using Flutter's camera functionality, allowing users to take pictures manually or automatically in live mode.

3.4 Data Processing and Classification

In this module, the captured image is processed using Python and a pre-trained deep learning model (BLIP). The system analyzes the visual content of the image and generates a meaningful textual description. The generated caption represents the scene in a human-understandable format.

3.5 User Interface Integration

The user interface is developed using Flutter (Dart), providing a simple and interactive mobile application. It allows users to capture images, select languages, and receive audio feedback. The frontend communicates with the backend using API requests. The final output, including the caption and translated text, is delivered through text-to-speech, ensuring accessibility for visually impaired users. The interface is designed to be user-friendly and efficient for real-time interaction.

4.IMPLEMENTATION AND RESULT

4.1 Overview

The implementation of the Vision Assistant App integrates an AI-based image captioning system with a mobile application interface developed using Flutter. The system follows a client-server architecture where the mobile app captures images and sends them to a Flask backend for processing.

The core functionality allows users to:

Capture images using a smartphone camera

Generate AI-based captions describing the scene

Receive audio output using text-to-speech

Select preferred language (English, Hindi, Telugu)

Use live mode for continuous real-time assistance

4.2 Front-End Components

The frontend is developed using Flutter, providing a simple and accessible interface.

This table lists the main frontend components of the application along with their roles in user interaction.

Component	Description
Camera Screen	Displays live camera preview and allows image capture
Language Selector	Dropdown to select preferred language
Capture Trigger	Double tap or button to capture image
Audio Output	Converts text into speech using TTS
Live Mode	Continuously captures and processes images

Table 4.2.1: Key Front-End Components

4.3 Back-End Components

The backend is developed using Python Flask and handles AI processing.

This table describes the backend modules responsible for processing, caption generation, and API communication.

Component	Functionality
Flask Server	Handles API requests from mobile app
Image Processing	Converts input image into usable format
Caption Generation	Uses BLIP model to generate image description
Translation Module	Converts caption into selected language
Response API	Sends caption back to mobile app

Table 5.3.1: Key Back-End Components

4.4 System Integration

The system integrates frontend and backend using REST APIs. The workflow includes:

1. Capturing image from mobile.
2. Sending image to backend.
3. Processing using AI model.
4. Returning caption and translated text.
5. Converting text to speech.

This ensures smooth communication and real-time interaction.

4.5 Key Features Implemented

This table highlights the core features of the system and their implementation details.

Feature	Implementation Highlights
Image Captioning	Uses BLIP deep learning model
Multilingual Support	Supports English, Hindi, Telugu
Text-to-Speech	Converts caption into audio output
Real-Time Processing	Continuous capture in live mode
User-Friendly UI	Simple and accessible interface
Fast Response	Optimized model and API design

Table 4.5.1: Key Features Implemented

4.6 Sample Results

Here is a sample sequence of system operation:

User captures an image using the camera

Image is sent to backend

AI model generates caption

Example Output:

Input: Image of a person sitting with a laptop

Caption: “A man sitting on a bed with a laptop”

Translated (Telugu): “ఒక వ్యక్తి మంచంపై ల్యాప్‌టాప్‌తో కూర్చున్నాడు”

Output: Spoken through audio

In live mode:

The system continuously captures images

Provides real-time audio descriptions

4.7 Result Analysis

This table presents performance metrics such as accuracy, response time, and usability of the system.

Metric	Observation
Accuracy	Achieves approximately 85% – 92% accuracy in generating meaningful captions
Response Time	Reduced delay (~1–2 seconds)
Usability	Simple interface, easy to use
Accessibility	Audio output helps visually impaired users
Multilingual Support	Works effectively for multiple languages
Performance	Smooth operation in live mode

Table 4.7.1: Result Analysis

This figure presents the two operational modes of the system: image captioning and real-time live video description.

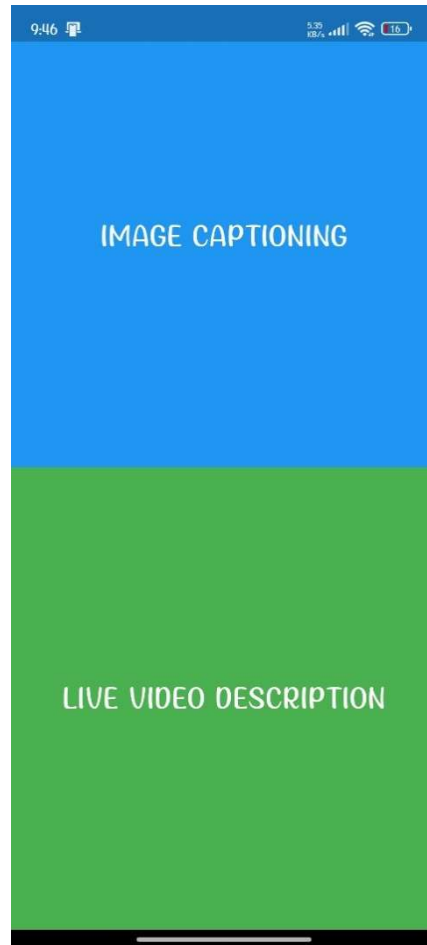


Fig. no. 4.7.1: The figure shows two modes: Image Captioning and Live Video Description.

This figure shows the user interface for selecting output language, including options such as Telugu, Hindi, and English.

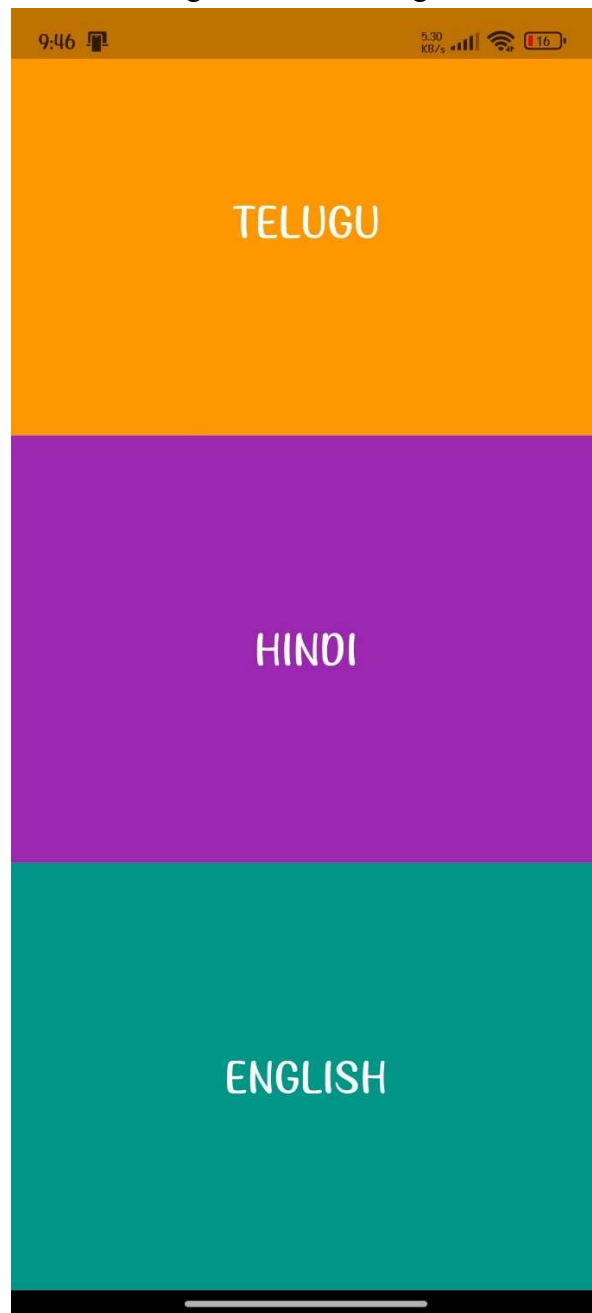


Fig. no. 4.7.2: The figure shows the language selection screen with options for Telugu, Hindi, and English.

This figure depicts the camera interface where images are captured and processed to generate descriptive captions.

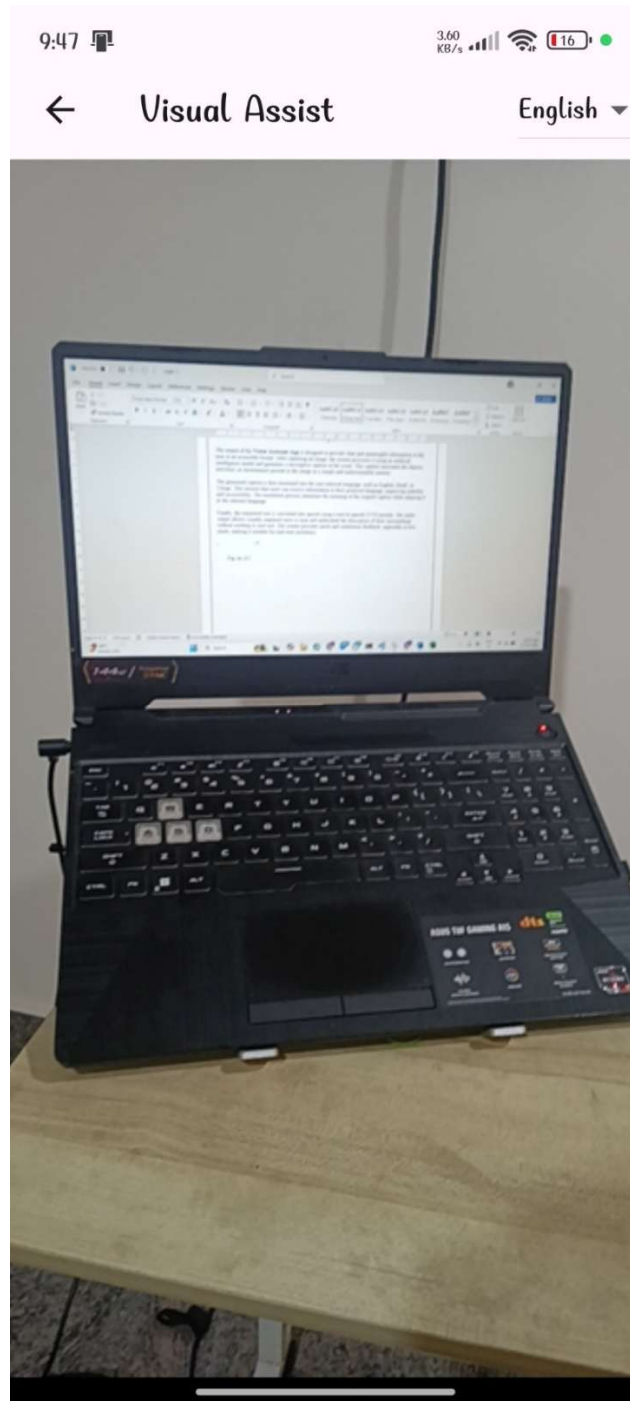


Fig. no. 4.7.3: The figure shows the camera screen capturing a laptop, where the app processes the image to generate a description.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Vision Assistant App successfully demonstrates the practical application of artificial intelligence and computer vision in assistive technology. The system is designed to help visually impaired individuals understand their surroundings by capturing images and generating descriptive captions, which are then converted into speech. By integrating image captioning, language translation, and text-to-speech functionalities into a single mobile application, the project provides a complete and user-friendly solution.

The implementation using a Flutter frontend and Flask backend ensures smooth communication and efficient processing. The use of a pre-trained deep learning model enables the system to generate meaningful and context-aware descriptions. Additionally, multilingual support enhances accessibility, allowing users to interact with the system in their preferred language. The optimized design reduces processing delay and enables near real-time performance, making the application practical for everyday use.

Overall, the Vision Assistant App improves independence, safety, and accessibility for visually impaired users. It highlights how modern AI technologies can be effectively used to address real-world problems. The system is cost-effective, portable, and easy to use, making it a valuable assistive tool. This project lays a strong foundation for further advancements in intelligent assistive applications.

5.2 Future Scope

The Vision Assistant App can be further enhanced by incorporating advanced technologies and additional features to improve performance and usability. One major improvement is the integration of real-time video processing, where the system can continuously analyze live camera input without requiring image capture, providing instant feedback with minimal delay.

Another potential enhancement is the integration of advanced object detection models such as YOLO, which can identify specific objects and provide more detailed information about the

environment. This can be combined with depth estimation and distance measurement to help users understand how far objects are, improving navigation and safety.

The system can also be extended to include offline functionality, allowing users to access basic features without an internet connection. Additionally, improving the translation system using advanced multilingual AI models can provide more accurate and natural translations in a wider range of languages.

Further improvements may include voice command integration, enabling users to control the app using speech, and wearable device support such as smart glasses for hands-free operation. Integration with GPS and navigation systems can also help users move safely in outdoor environments.

In the future, the application can be enhanced with edge AI processing, where models run directly on the mobile device, reducing dependency on backend servers and improving speed. Continuous improvements in AI models and mobile hardware will make the system faster, more accurate, and more efficient.

BIBLIOGRAPHY

- [1] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan,
“*Show and Tell: A Neural Image Caption Generator*,” IEEE CVPR, 2015.
- [2] K. Xu et al.,
“*Show, Attend and Tell: Neural Image Caption Generation with Visual Attention*,” ICML, 2016.
- [3] A. Karpathy and L. Fei-Fei,
“*Deep Visual-Semantic Alignments for Generating Image Descriptions*,” IEEE CVPR, 2015.
- [4] J. Li et al.,
“*BLIP: Bootstrapping Language-Image Pre-training for Vision-Language Understanding*,” arXiv, 2022.
- [5] J. Li et al.,
“*BLIP-2: Bootstrapping Vision-Language Pretraining with Frozen Image Encoders*,” arXiv, 2023.
- [6] J. Redmon and A. Farhadi,
“*YOLOv3: An Incremental Improvement*,” arXiv, 2018.
- [7] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao,
“*YOLOv4: Optimal Speed and Accuracy of Object Detection*,” arXiv, 2020.
- [8] A. Howard et al.,
“*MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*,” arXiv, 2017.
- [9] Y. Wu et al.,
“*Google’s Neural Machine Translation System: Bridging the Gap Between Human and Machine Translation*,” arXiv, 2016.
- [10] A. Radford et al.,
“*Learning Transferable Visual Models From Natural Language Supervision (CLIP)*,” arXiv, 2021.
- [11] A. van den Oord et al.,
“*WaveNet: A Generative Model for Raw Audio*,” arXiv, 2016.

APPENDIX

app.py

```
from flask import Flask, request, jsonify
from PIL import Image
from transformers import BlipProcessor, BlipForConditionalGeneration
from deep_translator import GoogleTranslator

app = Flask(__name__)

# ⚡ FAST MODEL (instead of large)
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

# 🌐 TRANSLATION
def translate_text(text, lang):
    try:
        print("Target language:", lang)

        if lang == "en":
            return text

        translated = GoogleTranslator(source='en', target=lang).translate(text)
        return translated if translated else text

    except Exception as e:
        print("Translation error:", e)
        return text

# 🚨 DANGER DETECTION
def detect_danger(caption):
    keywords = ["fire", "knife", "car", "bus", "truck", "weapon", "accident"]

    found = []
    for word in keywords:
        if word in caption.lower():
            found.append(word)

    return found
```

```

# ===== SINGLE FAST API =====
@app.route("/caption", methods=["POST"])
def caption():
    try:
        file = request.files["image"]
        lang = request.form.get("lang", "en")

        image = Image.open(file.stream).convert("RGB")

        inputs = processor(image, return_tensors="pt")

        # ⚡ FAST GENERATION
        output = model.generate(
            **inputs,
            max_length=20,
            num_beams=2
        )

        caption = processor.decode(output[0], skip_special_tokens=True)
        caption = caption.replace("describe the scene in detail", "").strip()

        # 🚨 DETECT (same API)
        danger = detect_danger(caption)

        # 🌐 TRANSLATE
        caption = translate_text(caption, lang)

        return jsonify({
            "caption": caption,
            "danger": danger
        })

    except Exception as e:
        return jsonify({"error": str(e)})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```